

You can observe that measurements specified in the INTO clause come under the autogen or long-term policy to keep low precision data forever or for a longer period, and the FROM clause is applied on measurements with a policy whose duration is smaller than that of the INTO clause.

With the help of the above two features, a high volume of data can be efficiently reduced to low volumes periodically.

InfluxDb is the right choice for storing high precision data like hundreds of thousands of points per second, as it has unique downsampling support. Therefore, many IoT and DevOps frameworks and solutions are adopting Influx in place of traditional alternatives. Here is a small listing of use cases:

- Plugin for monitoring of internal metrics of HiveMQ
- For storing tenant data from various devices in SiteWhere
- Apache Spark, Kafka integrations for feeding data or storing the results
- OpenHAB add-ons
- Resource usage monitoring by Kubernetes

Let's now take a quick look at Telegraf and its relation to InfluxDB.

Telegraf

Telegraf is a data gathering and feeding agent for InfluxDb, with a rich set of plugins for collecting and reporting metrics for different use cases like system performance and monitoring systems. Once Telegraf is launched directly through its binary or by using service managers, it activates input and output plugins as specified in the supplied configuration file. The default configuration file is stored in `/etc/telegraf/telegraf.conf`; you can modify it or provide a custom `config` file to the binary.

By default, input plugins related to system resources collect statistics like CPU, memory, diskIO, kernel, swapping and process related parameters. These are stored in respective measurements under the database named *telegraf* using InfluxDb output plugin. To check this, open the Influx shell in CLI or Web mode, select *telegraf* as the database and list out all measurements and series using 'SHOW MEASUREMENTS', 'SHOW SERIES', respectively.

Let's consider the usage of Telegraf in an IoT application. If end devices or gateways need to store the values into a database on the remote server side, supporting HTTP APIs can cause overheads all the time due to frequent updates. To minimise this, Telegraf comes with the MQTT service plugin, where devices publish data to the MQTT broker in a lightweight manner. Here, Telegraf acts like a subscriber receiving all the matching data, and stores the collected points in InfluxDb. For this, publishing devices should take care that the MQTT payload should be in the form of InfluxDb Line protocol. To enable this integration

of MQTT and InfluxDb, edit the *telegraf* configuration as follows. Locate *inputs.mqtt_consumer* in *telegraf.conf* and uncomment the lines as follows:

```
[inputs.mqtt_consumer]
qos=0
topics = [
    "telegraf/weather/temperature",
    "telegraf/weather/humidity",
    "telegraf/smarthome/#",
]
data_format = "influx"
```

You may fill a few more fields under this section optionally. Now, Telegraf collects payloads published with specified topics and stores them in the *telegraf* database by default. This has an added advantage of batch inputs, to minimise the usage of HTTP requests by merging multiple points falling under a particular interval into a single HTTP transaction.

So one can consider MQTT broker, Telegraf and InfluxDb on the same server node or in a cluster of different nodes with good network support; devices behind constrained networks publish data using the lightweight MQTT protocol.

Telegraf also comes with the MQTT output plugin to publish a few metrics to a broker residing on a remote machine. These can be used on Linux compatible targets to collect the following types of data:

- On board system metrics
- Sensor data collected through input plugin 'sensors'
- Data from nearby devices collected through TCP, UDP service plugins

Please check Telegraf documentation for a complete list of plugins and their usage. Once data is collected and stored, a dashboard has the role of visualisation of data. Chronograf is an immature visualisation tool from the TICK stack, but Grafana is a better choice with good integration for InfluxDb and many other databases. Apart from visualisation, if you wish to process the data, find the anomalies, schedule for alerts and do basic ETL jobs, Kapacitor is the right tool. 

References

- [1] <https://www.influxdata.com>
- [2] <http://opensourceforu.com/2016/12/introduction-influxdb-time-series-database/>

By: Rajesh Sola

The author is a faculty member of C-DAC's Advanced Computing Training School, and an evangelist in the embedded systems and IoT domains. You can reach him at rajeshsola@gmail.com.